

CS 550 Operating Systems

Spring 2026

Condition variables

POSIX threads API: pthread

- Implementations of the API are available on many Unix-like POSIX-conformant operating systems.
- There are around 100 Pthreads procedures, all prefixed "pthread_" and they can be categorized into four groups:
 - Thread management - creating, joining threads etc.
 - Mutexes
 - Condition variables
 - Semaphores

The need of checking conditions

- There are many cases where a thread wishes to check whether a condition is true before continuing its execution.

```
void *child(void *arg) {  
    printf("child\n");  
    // XXX how to indicate we are done?  
    return NULL;  
}  
  
int main(int argc, char *argv[]) {  
    printf("parent: begin\n");  
    pthread_t c;  
    Pthread_create(&c, NULL, child, NULL); // create child  
    // XXX how to wait for child?  
    printf("parent: end\n");  
    return 0;  
}
```

parent: begin
child
parent: end

Option 1: shared variable + spin-wait

```
1  volatile int done = 0;
2
3  void *child(void *arg) {
4      printf("child\n");
5      done = 1;
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     printf("parent: begin\n");
11     pthread_t c;
12     Pthread_create(&c, NULL, child, NULL); // create child
13     while (done == 0)
14         ; // spin
15     printf("parent: end\n");
16     return 0;
17 }
```

Option 2: condition variable

- A condition variable allows one thread to inform other threads about changes in the state of certain shared resource, and allows the other threads to wait (block) for such notification.
- To declare a conditional variable:

```
pthread_cond_t c;
```

- Two operations are associated with a conditional variable `wait()` and `signal()`:

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);  
pthread_cond_signal(pthread_cond_t *c);
```

wait () operation of condition variable

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
```

- Puts the calling thread to sleep.
- The calling thread will sleep till another thread signals it to wake up.
- `pthread_cond_wait()` assumes the mutex `m` is locked when the `pthread_cond_wait()` is called. It puts the calling thread to sleep and releases the mutex `m` atomically.
 - Why do we need a mutex here?
- The calling thread is woken up by another thread performing `pthread_cond_signal()` on the same CV.
- After being woken up, the calling thread needs to re-acquire the lock before returning from `pthread_cond_wait()`.

signal () operation of condition variable

```
pthread_cond_signal(pthread_cond_t *c);
```

- Signals the thread that is (or one of the threads that are) waiting on the conditional variable **c** to wake up.

Waiting child thread using CV

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}
```

- Main thread runs first after pthread_create() ...
- Child thread runs first after pthread_create() ...



Remove the mutex m ?

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

void thr_join() {
    Pthread_mutex_lock(&m);
P1 while (done == 0)
P2     Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_exit() {
    Pthread_mutex_lock(&m);
C1 done = 1;
C2 Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}
```

Can you think of an execution scenario which will cause a problem?

Remove the mutex m ?

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

void thr_join() {
    pthread_mutex_lock(&m);
P1 while (done == 0)
P2     Pthread_cond_wait(&c, &m);
    pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_exit() {
    pthread_mutex_lock(&m);
C1 done = 1;
C2 Pthread_cond_signal(&c);
    pthread_mutex_unlock(&m);
}
```

P1 → C1 → C2 → P2



Remove the state variable done?

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

void thr_join() {
    P1 Pthread_mutex_lock(&m);
while (done == 0)
    P2 Pthread_cond_wait(&c, &m);
    P3 Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_exit() {
    C1 Pthread_mutex_lock(&m);
done = 1;
    C2 Pthread_cond_signal(&c);
    C3 Pthread_mutex_unlock(&m);
}
```

C1 → C2 → C3 → P1 → P2



The producer/consumer problem

- Imagine one or more **producer threads** and one or more **consumer threads**. Periodically
 - **Producers** generate data items and place them in a buffer (until the buffer is full).
 - **Consumers** grab said items from the buffer and consume them in some way (if the buffer is not empty).
- This arrangement occurs in many real systems.
 - For example, in a multi-threaded web server, a producer puts HTTP requests into a work queue (i.e., the bounded buffer); consumer threads take requests out of this queue and process them.

Dealing producer/consumer problem with CV

Assuming a single slot buffer (just a integer)

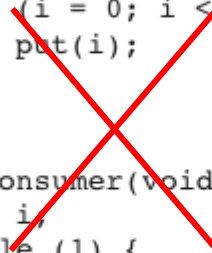
```
int buffer;
int count = 0; // initially, empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}
```

```
void *producer(void *arg) {
    int i;
    int loops = (int) arg;
    for (i = 0; i < loops; i++) {
        put(i);
    }
}

void *consumer(void *arg) {
    int i;
    while (1) {
        int tmp = get();
        printf("%d\n", tmp);
    }
}
```



A broken solution

```
int buffer;
int count = 0; // initially, empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}

-----

cond_t cond;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&mutex); // p1
        if (count == 1) // p2
            pthread_cond_wait(&cond, &mutex); // p3
        put(i); // p4
        pthread_cond_signal(&cond); // p5
        pthread_mutex_unlock(&mutex); // p6
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&mutex); // c1
        if (count == 0) // c2
            pthread_cond_wait(&cond, &mutex); // c3
        int tmp = get(); // c4
        pthread_cond_signal(&cond); // c5
        pthread_mutex_unlock(&mutex); // c6
        printf("%d\n", tmp);
    }
}
```

Two consumer threads: **A** & **B**; one producer thread: **X**

c1 (A)→c2 (A)→c3 (A)→A sleeps

→p1 (X)→p2 (X)→p4 (X)→p5 (X)→p6 (X)→p1 (X)→p2 (X)→p3 (X)→X sleeps

→A wakes up at c3 as a result of p5 by X, but before A can get the lock “mutex”, B comes in and gets the lock

→c1(B)→c2(B)→c4(B) (The product by produced by X is consumed here by B!) →c5(B)→C6(B)

→A gets the lock and returns from c3→c4(A), A has no product to consume or A consumes a product which has been consumed by B before!

A broken solution

```

int buffer;
int count = 0; // initially, empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}

-----

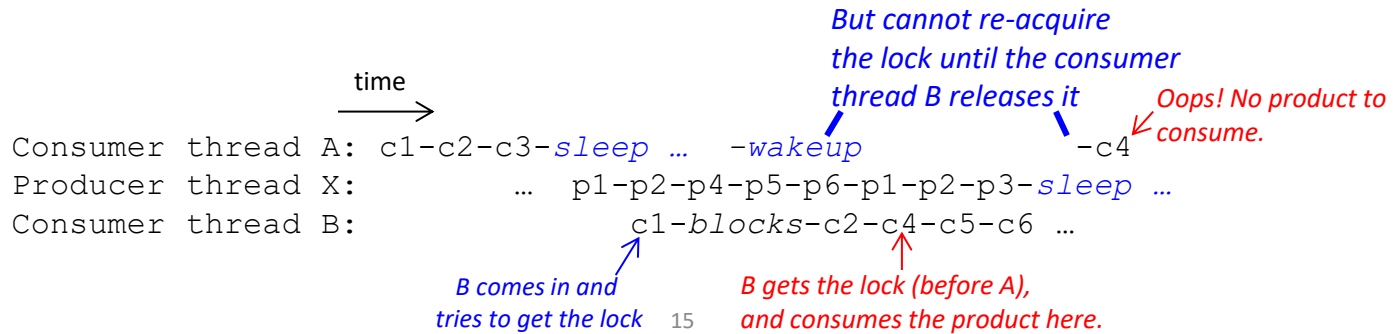
cond_t cond;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&mutex); // p1
        if (count == 1) // p2
            pthread_cond_wait(&cond, &mutex); // p3
        put(i); // p4
        pthread_cond_signal(&cond); // p5
        pthread_mutex_unlock(&mutex); // p6
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        pthread_mutex_lock(&mutex); // c1
        if (count == 0) // c2
            pthread_cond_wait(&cond, &mutex); // c3
        int tmp = get(); // c4
        pthread_cond_signal(&cond); // c5
        pthread_mutex_unlock(&mutex); // c6
        printf("%d\n", tmp);
    }
}

```

Look from another way:



A better solution, but ...

```

int buffer;
int count = 0; // initially, empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}

-----

cond_t cond;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);           // p1
        while (count == 1)                    // p2
            Pthread_cond_wait(&cond, &mutex); // p3
        put(i);                               // p4
        Pthread_cond_signal(&cond);           // p5
        Pthread_mutex_unlock(&mutex);         // p6
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);           // c1
        while (count == 0)                    // c2
            Pthread_cond_wait(&cond, &mutex); // c3
        int tmp = get();                     // c4
        Pthread_cond_signal(&cond);           // c5
        Pthread_mutex_unlock(&mutex);         // c6
        printf("%d\n", tmp);
    }
}

```

*Could wake up either the
consumer thread 2 or the
producer thread*

*If thread 2 is the one
woken up ...*

time →

```

Consumer thread1: c1-c2-c3-sleep ... .. -c2-c4-c5-c6-c1-c2-c3-sleep
Consumer thread2: c1-c2-c3-sleep ... .. -c2-c3-sleep
Producer thread:  p1-p2-p4-p5-p6-p1-p2-p3-sleep

```


Using two conditional variables

```
int buffer;
int count = 0; // initially, empty

void put(int value) {
    assert(count == 0);
    count = 1;
    buffer = value;
}

int get() {
    assert(count == 1);
    count = 0;
    return buffer;
}
```

```
cond_t empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 1)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```

The final working solution

```
int buffer[MAX];
int fill = 0;
int use = 0;
int count = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
    count++;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    count--;
    return tmp;
}

cond_t empty, fill;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);           // p1
        while (count == MAX)                  // p2
            Pthread_cond_wait(&empty, &mutex); // p3
        put(i);                               // p4
        Pthread_cond_signal(&fill);           // p5
        Pthread_mutex_unlock(&mutex);         // p6
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);           // c1
        while (count == 0)                   // c2
            Pthread_cond_wait(&fill, &mutex); // c3
        int tmp = get();                     // c4
        Pthread_cond_signal(&empty);         // c5
        Pthread_mutex_unlock(&mutex);         // c6
        printf("%d\n", tmp);
    }
}
```

```
#include <pthread.h>
```

```
int pthread_cond_init(pthread_cond_t *restrict cond,  
                      const pthread_condattr_t *restrict attr);
```

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Both return: 0 if OK, error number on failure

```
#include <pthread.h>
```

```
int pthread_cond_wait(pthread_cond_t *restrict cond,  
                      pthread_mutex_t *restrict mutex);
```

```
int pthread_cond_timedwait(pthread_cond_t *restrict cond,  
                           pthread_mutex_t *restrict mutex,  
                           const struct timespec *restrict tspr);
```

Both return: 0 if OK, error number on failure

```
#include <pthread.h>
```

```
int pthread_cond_signal(pthread_cond_t *cond);
```

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Both return: 0 if OK, error number on failure